Program: spectrumify
Bytes: ~12,000 lines
Sinclair ZX Spectrum
48K BASIC

Spectrumify

DEVELOPMENT REPORT

Estudio comparativo de una aplicación implementada
en 7 lenguajes de programación
(Sinclair BASIC 1982 → ARM64 Assembly 2026)

1	7	~25	~12K	9	4
DÍA	LENGUAJES	COMMITS	LÍNEAS	DOCUMENTOS	AUDITORÍAS

> JGF (José Gómez Freelance)
> spectrumify

12 de abril de 2026 — Sesión única
Informe generado: 12/04/2026

Python → C → Pascal → ARM64 ASM → LISP → Sinclair BASIC → Z80 ASM

> READY._

> 1. RESUMEN DEL PROYECTO

Spectrumify convierte imágenes fotográficas a pixel-art con paleta ZX Spectrum (16 colores) y las exporta como HTML, SCR y GIF animado. El proyecto comenzó como tres scripts Python y evolucionó hasta una aplicación de escritorio implementada en 7 lenguajes de programación en una sesión única de desarrollo.

1	7	~12K	9	4
DÍA	LENGUAJES	LÍNEAS	DOCS	AUDITORÍAS

MÉTRICA	VALOR
Periodo de desarrollo	12 de abril de 2026 (sesión única)
Lenguajes implementados	7 (Python, C, Pascal, ARM64 ASM, LISP, Sinclair BASIC, Z80 ASM)
Commits totales	~25
Líneas de código totales	~12,000
Documentos técnicos	9
Auditorías de seguridad	4 (C, Pascal, LISP, SDL2)
Co-autoría	100% con Claude Opus 4.6 (1M context)

> 2. EVOLUCIÓN DEL PROYECTO

Fig. 1 — Evolución del proyecto en 9 fases

> 3. TAMAÑO DEL EJECUTABLE

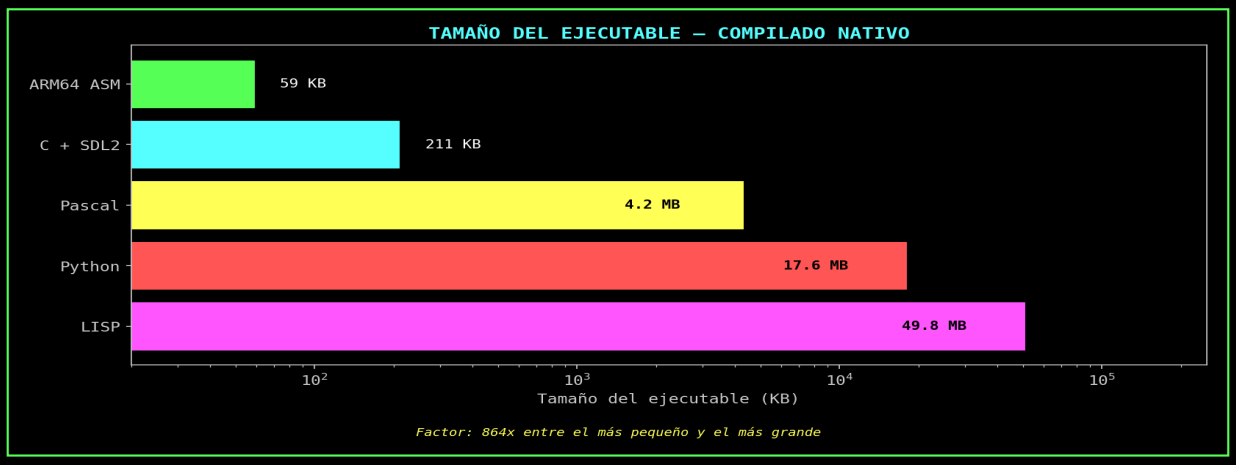

Fig. 2 — Tamaño del ejecutable compilado nativo (escala logarítmica)

ARM64 ASM es el más pequeño (59 KB) porque no incluye ningún runtime ni librería embebida. LISP es el mayor (51 MB) porque SBCL embebe su compilador y runtime completo. Factor: 864x entre ambos extremos.

> 4. LÍNEAS DE CÓDIGO

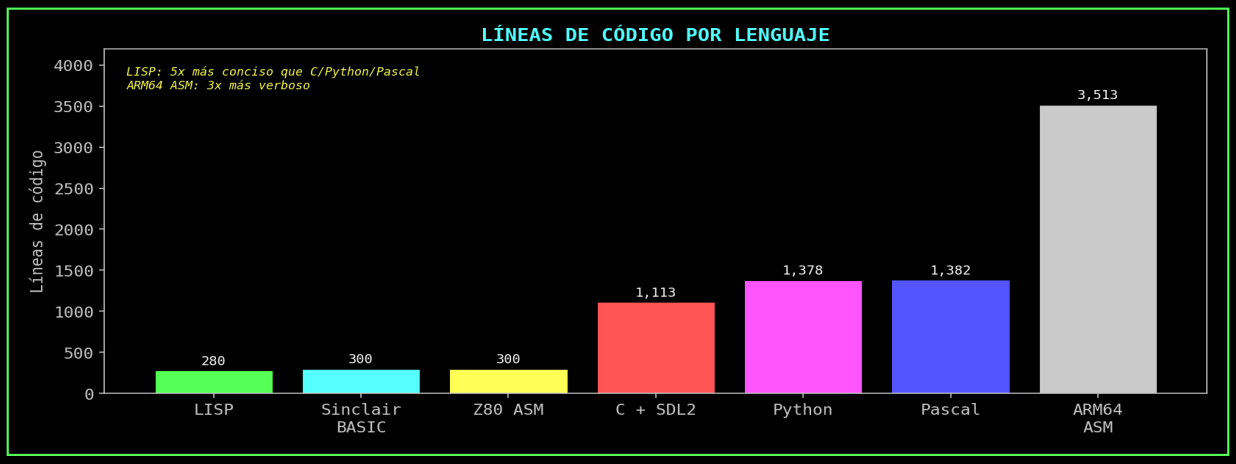

Fig. 3 — Líneas de código por lenguaje (7 implementaciones)

LISP es 5x más conciso que C/Python/Pascal para la misma funcionalidad. ARM64 ASM requiere 3x más líneas que C para un resultado equivalente.

> 5. VELOCIDAD DE CONVERSIÓN

Imagen de prueba: foto.png (1536×1024) → 153×102 celdas, 16 colores ZX Spectrum.

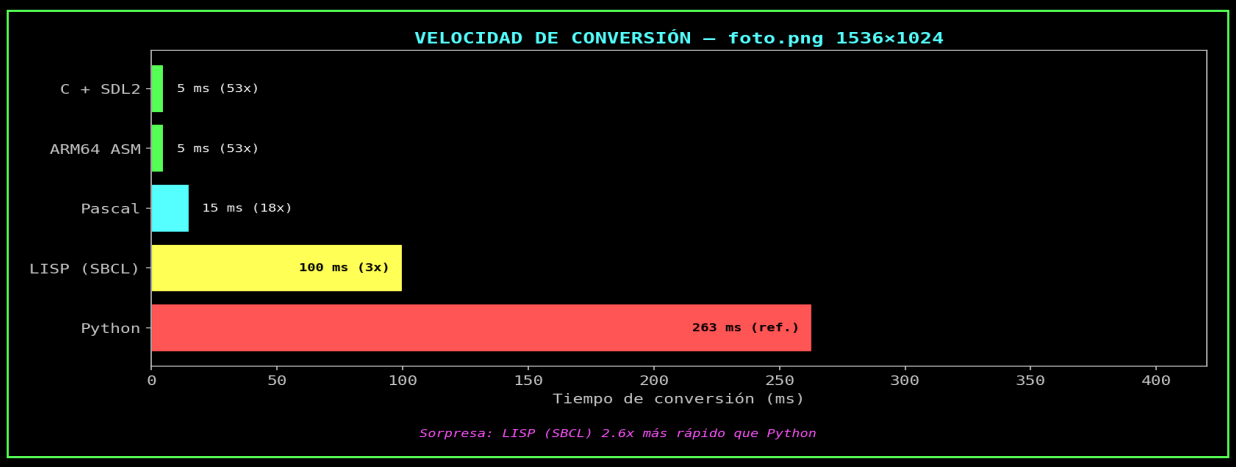

Fig. 4 — Velocidad de conversión (menor = mejor)

VERSIÓN	TIEMPO	FACTOR VS PYTHON
C + SDL2	~5 ms	53x más rápido
ARM64 ASM	~5 ms	53x más rápido
Pascal	~15 ms	18x más rápido
LISP (SBCL)	~100 ms	2.6x más rápido
Python	263 ms	1x (referencia)

> 6. USO DE RAM

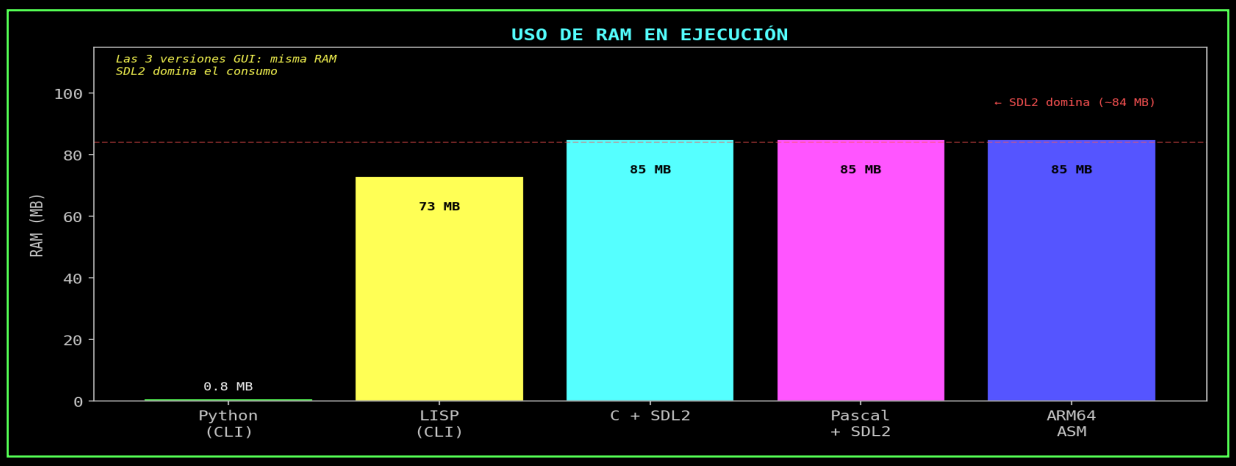

Fig. 5 — Uso de RAM en ejecución

> 7. TIEMPO DE ARRANQUE

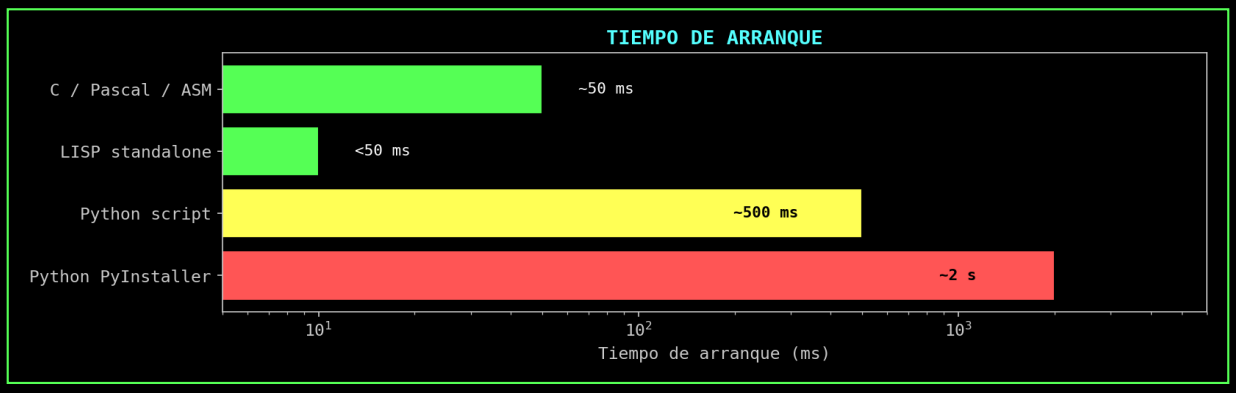

Fig. 6 — Tiempo de arranque por versión

> 8. BUGS DURANTE EL DESARROLLO

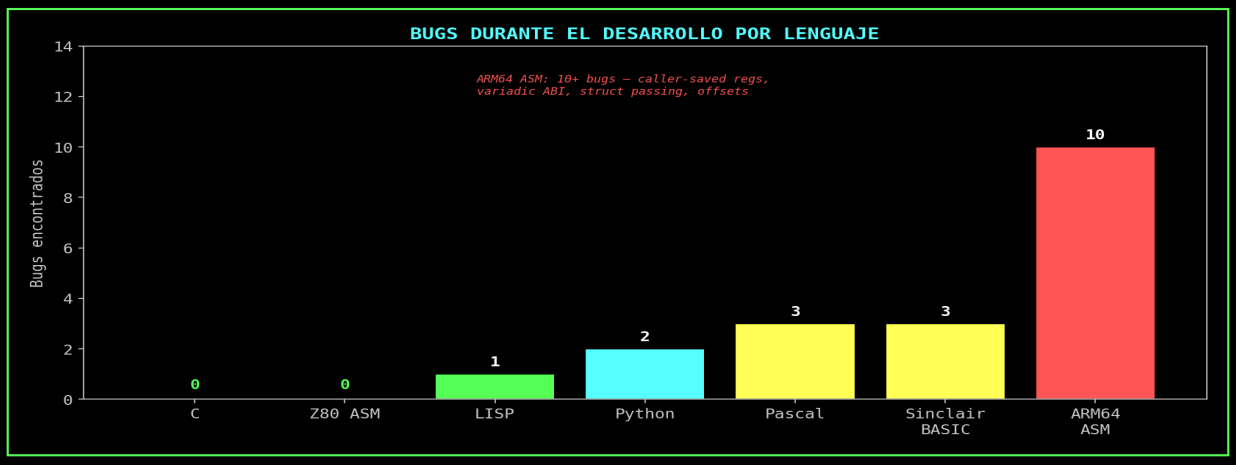

Fig. 7 — Bugs por lenguaje: correlación inversa con nivel de abstracción

LENGUAJE	BUGS	CAUSA PRINCIPAL
C	0	Ningún bug
Z80 ASM	0	Verificado con pasmo
LISP	1	Export de main no exportado
Python	2	SDL2/tkinter conflicto, preview scaling
Pascal	3	Comentarios {}, variables inline, controles
Sinclair BASIC	3	Colisiones de números de línea
ARM64 ASM	10+	Caller-saved regs, variadic ABI, offsets, struct passing

> 9. CONSISTENCIA DE SALIDA

Todas las versiones producen resultados idénticos, byte a byte:

FORMATO	PYTHON	C	PASCAL	ARM64 ASM	LISP
HTML (153×102)	703,339 B	703,339 B	703,339 B	703,339 B	703,339 B
SCR	6,912 B	6,912 B	6,912 B	6,912 B	6,912 B

El lenguaje no importa para el resultado final. Los cinco producen el mismo HTML de 703 KB.

> 10. RESUMEN COMPARATIVO

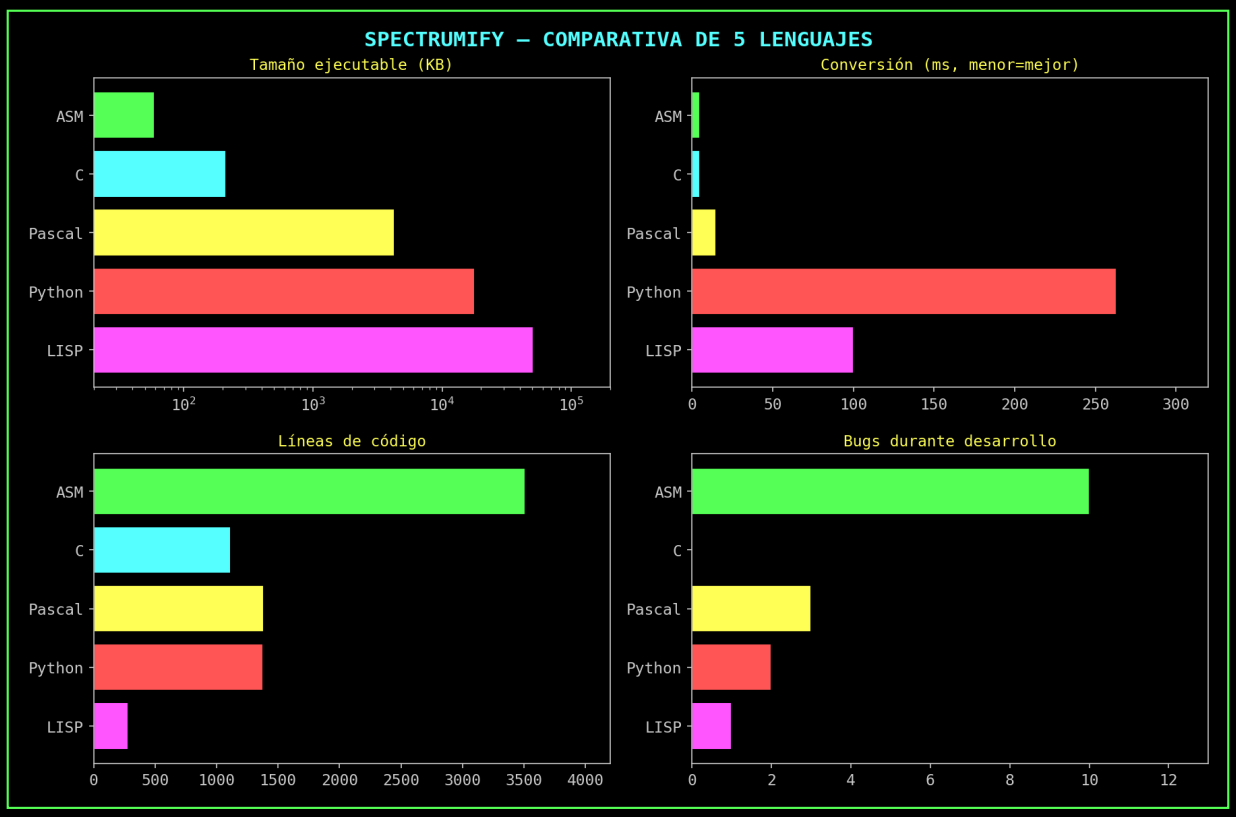

Fig. 8 — Comparativa de 5 lenguajes: tamaño, velocidad, líneas, bugs

> 11. DESCUBRIMIENTOS TÉCNICOS

>> Apple ARM64 Variadic ABI

En macOS ARM64, las funciones variádicas (printf, fprintf, sprintf) reciben los argumentos variádicos en el STACK, no en registros X1-X7. Esto difiere del AAPCS64 estándar usado en Linux ARM64. No está documentado explícitamente y requirió debugging empírico.

```
// INCORRECTO en Apple ARM64:
mov     w1, #42
bl      _printf       // NO lee w1 para args variádicos

// CORRECTO en Apple ARM64:
sub     sp, sp, #16
str     x1, [sp]      // arg variádico en STACK
bl      _printf
add     sp, sp, #16
```

>> SDL2/tkinter crash en macOS

pygame (SDL2) y tkinter (Tk) no pueden coexistir en el mismo proceso en macOS. SDL2 toma control de NSApplication y tkinter intenta crear su propia instancia, causando un crash con NSInvalidArgumentException. Solución: usar osascript (AppleScript nativo) para diálogos de archivo.

>> CFFI struct passing en ARM64

CFFI en SBCL no puede pasar structs (NSRect, CGRect) correctamente en ARM64 macOS. La ABI de Apple requiere pasar structs pequeños en registros, pero CFFI no implementa esta convención. Esto hace imposible crear ventanas GUI desde Common Lisp en Apple Silicon vía Cocoa FFI o SDL2.

>> macOS 26.4 mata Wish (Tk)

El Tk 8.5.9 del sistema tiene una firma de código que macOS 26.4 rechaza con SIGKILL (Code Signature Invalid). Afecta a cualquier programa que use ltk o tkinter desde procesos no firmados.

>> Hex corto para paleta ZX

Los 16 colores del ZX Spectrum tienen componentes RGB con dígitos hex repetidos (AA, 55, FF, 00), por lo que todos son abreviables: #AA0000 → #A00. Ahorro: 3 bytes por celda, ~46 KB en un export típico (~6% de reducción).

> 12. LIBRERÍAS CREADAS

>> GIF Encoder para Free Pascal (gifwriter.pas)

Encoder GIF mínimo con compresión LZW completa. Frame único, paleta indexada de hasta 256 colores. ~180 líneas de Pascal puro, sin dependencias. Reutilizable como unit independiente.

>> BMP Loader en ARM64 ASM (bmploader.s)

Cargador de archivos BMP 24-bit en ensamblador ARM64 puro. Lee el header BMP byte a byte (evitando problemas de alineación), convierte BGR a RGB, y maneja BMP bottom-up y top-down.

> 13. AUDITORÍAS DE SEGURIDAD

Se auditaron 12 componentes externos en busca de: conexiones de red, telemetría, ejecución de comandos, acceso a variables de entorno, código ofuscado y exfiltración de datos.

COMPONENTE	TIPO	VEREDICTO
stb_image.h	C header-only	LIMPIO
stb_image_write.h	C header-only	LIMPIO
msf_gif.h	C header-only	LIMPIO
SDL2	Librería sistema	LIMPIO
SDL2-for-Pascal	Pascal bindings	LIMPIO
Free Pascal (FPC)	Compilador	SEGURO
SBCL	Compilador CL	SEGURO
Quicklisp	Gestor paquetes	PRECAUCIÓN (HTTP)
opticl	CL imagen	LIMPIO
CFFI	CL FFI	LIMPIO
pasmo	Z80 assembler	SEGURO
ltk	CL Tk bindings	NO USABLE (Wish crash)

> 14. CONCLUSIONES

>> Lo que confirmó la comparativa teórica

> C es el mejor equilibrio eficiencia/practicidad para apps de escritorio.
> Python es el más rápido de desarrollar pero el más lento en ejecución.
> LISP es el más conciso algorítmicamente (280 líneas).
> Ensamblador moderno no tiene ganancia práctica sobre C -O2.

>> Lo que no predijimos

> LISP 2.6x más rápido que Python (SBCL compila a nativo).
> Las tres versiones SDL2 consumen la misma RAM (85 MB) — SDL2 domina.
> ARM64 ASM tuvo 10+ bugs vs 0 en C — diferencia mayor de lo esperado.
> GUI en LISP resultó imposible en Apple Silicon (3 caminos bloqueados).
> El variadic ABI de Apple es diferente al estándar AAPCS64.

>> Cifras clave

MÉTRICA	VALOR
Periodo de desarrollo	1 día (sesión única, 12 de abril de 2026)
Lenguajes	7 implementaciones completas
Producción total	~12,000 líneas de código + 9 documentos técnicos
Auditorías	4 auditorías de seguridad (12 componentes)
Librerías creadas	2 (GIF Encoder Pascal + BMP Loader ARM64)
Co-autoría	100% con Claude Opus 4.6 (1M context)
Consistencia	703,339 bytes HTML idénticos en 5 versiones

Spectrumify demuestra que el lenguaje importa menos de lo que se cree para el resultado final (todos producen el mismo HTML de 703 KB), pero importa enormemente para la experiencia del desarrollador. Las 280 líneas de LISP y las 3,513 de ARM64 ASM hacen exactamente lo mismo — la diferencia es cuánto sufres para llegar ahí.

// Informe generado el 12/04/2026.
// Co-desarrollado con Claude Opus 4.6 (1M context).

// READY.